

ML-App Schritt für Schritt erklärt

Was geht hinein? Was passiert? Was kommt heraus?

1. Daten laden

Code: `df = pd.DataFrame(testdaten, columns=["text", "label"])`

Erklärung: Du hast vorher eine normale Python-Liste, zum Beispiel mit einem Text und dem passenden Label. `pd.DataFrame(...)` macht daraus eine Tabelle. Damit können pandas und später scikit-learn sauber arbeiten.

Beispiel: `["Ich habe Kopfschmerzen.", "symptom"]` -> Tabelle mit den Spalten text und label.

2. X und y festlegen

Code: `X = df["text"]`

`y = df["label"]`

Erklärung: X sind die Eingabedaten, die das Modell bekommt. y ist die bekannte richtige Antwort, also die Ground Truth.

Beispiel: `X = "Ich habe Kopfschmerzen." | y = "symptom"`

Merksatz: X = Eingabe, y = richtige Antwort.

3. Train/Test-Split

Code: `train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)`

Erklärung: Die Daten werden in Training und Test aufgeteilt. 70 % werden zum Lernen verwendet, 30 % bleiben für die finale Bewertung zurück.

random_state=42: Sorgt dafür, dass bei jedem Start dieselbe zufällige Aufteilung entsteht. Die Zahl 42 selbst ist nicht besonders wichtig.

stratify=y: Versucht, die Klassenverteilung in Train und Test ähnlich zu halten.

Merksatz: Trainingsdaten = Übungsaufgaben. Testdaten = Klassenarbeit.

4. Die Pipeline

Code: `Pipeline([("tfidf", TfidfVectorizer()), ("classifier", LogisticRegression())])`

Erklärung: Die Pipeline verbindet mehrere Schritte: Text -> TF-IDF -> Klassifikationsmodell.

TF-IDF: Ein ML-Modell kann mit Rohtext nicht direkt rechnen. TF-IDF wandelt Text in Zahlenvektoren um.

Beispiel: "Ich habe Kopfschmerzen" -> [0.0, 0.3, 0.8, ...]

5. Warum mehrere Modelle?

Erklärung: Du vergleichst Logistic Regression und LinearSVC, weil du vorher nicht sicher weißt, welches Modell auf deinen Daten besser funktioniert.

Merksatz: Nicht raten, sondern Modelle auf denselben Daten vergleichen.

6. Was ist C?

Code: "classifier__C": [0.1, 1, 10]

Erklärung: C steuert vereinfacht, wie stark sich das Modell an die Trainingsdaten anpassen darf.

C klein: Stärkere Regularisierung. Das Modell bleibt vorsichtiger und einfacher.

C groß: Schwächere Regularisierung. Das Modell kann sich stärker an die Trainingsdaten anpassen.

Regularisierung: Sie soll verhindern, dass das Modell die Trainingsdaten zu stark auswendig lernt.

Merksatz: Es gibt kein grundsätzlich bestes C. Deshalb testet GridSearch mehrere Werte.

7. Was ist ngram_range?

Code: "tfidf__ngram_range": [(1, 1), (1, 2)]

(1,1): Nur einzelne Wörter werden betrachtet.

(1,2): Zusätzlich werden Wortpaare betrachtet.

Beispiel: "starke Kopfschmerzen" -> einzelne Wörter plus "starke Kopfschmerzen".

Hinweis: Wortpaare wie "keine Schmerzen" können aussagekräftiger sein als einzelne Wörter.

8. Was ist parameter_grid?

Erklärung: parameter_grid ist die Liste aller Varianten, die GridSearch ausprobieren soll.

Beispiele: Logistic Regression mit C=0.1 und ngram=(1,1); Logistic Regression mit C=10 und ngram=(1,2); LinearSVC mit C=1 und ngram=(1,2).

Merksatz: GridSearch probiert alle definierten Kombinationen aus.

9. GridSearchCV

Code: GridSearchCV(pipeline, parameter_grid, cv=5, scoring="f1_macro")

pipeline: Beschreibt den Ablauf Text -> TF-IDF -> Klassifikator.

parameter_grid: Beschreibt, welche Modelle und Einstellungen getestet werden.

cv=5: Jede Kombination wird mit 5-Fold-Cross-Validation geprüft.

scoring=f1_macro: Jede Kombination wird anhand des Macro-F1 bewertet.

10. Was passiert bei Cross-Validation?

Erklärung: Die Trainingsdaten werden in 5 Teile geteilt. In jedem Durchlauf dienen 4 Teile zum Trainieren und 1 Teil zur Validierung.

Ablauf: Durchlauf 1: A ist Validation. Durchlauf 2: B ist Validation. Danach C, D und E.

Ergebnis: Für jeden Fold entsteht ein F1-Wert. Der Durchschnitt zeigt, wie stabil das Setup über mehrere Aufteilungen funktioniert.

11. Was bedeutet scoring='f1_macro'?

Erklärung: Macro-F1 bedeutet: Jede Klasse zählt gleich viel, unabhängig davon, wie oft sie vorkommt.

Beispiel: symptom, medikament und negation werden gleich gewichtet.

Merksatz: Besonders sinnvoll, wenn alle Klassen wichtig sind.

12. Was passiert bei grid_search.fit(X_train, y_train)?

Input: X_train = Trainings-Texte, y_train = richtige Labels.

Intern: GridSearch erzeugt alle Modell- und Parameterkombinationen, führt für jede Kombination Cross-Validation durch, berechnet F1, vergleicht die Ergebnisse und bestimmt das beste Setup.

Wichtig: X_test und y_test werden dabei nicht verwendet.

13. best_params_

Erklärung: best_params_ zeigt die beste gefundene Kombination aus Modell und Hyperparametern.

Beispiel: LogisticRegression, C=10, ngram_range=(1,2).

Bedeutung: Diese Kombination war im Durchschnitt der Cross-Validation am besten.

14. best_score_

Erklärung: best_score_ ist der durchschnittliche Cross-Validation-Score des besten Setups.

Beispiel: 0.84 bedeutet: Das beste Setup hatte auf den Trainingsdaten im Durchschnitt einen Macro-F1 von 0.84.

15. best_estimator_

Code: `best_model = grid_search.best_estimator_`

Erklärung: Das ist dein finales ausgewähltes Modell inklusive der besten TF-IDF-Einstellungen und Modellparameter.

Wichtig: GridSearchCV trainiert dieses beste Setup standardmäßig noch einmal auf allen Trainingsdaten.

16. Finale Test-Vorhersage

Code: `y_pred = best_model.predict(X_test)`

Input: `X_test` = Texte, die nicht für die Modellauswahl verwendet wurden.

Intern: Text -> TF-IDF -> bestes Modell -> Klasse.

Output: `y_pred` enthält die vorhergesagten Klassen.

17. Warum `y_test` mit `y_pred` vergleichen?

`y_test`: Die bekannte richtige Antwort.

`y_pred`: Die Antwort des Modells.

Beispiel: `y_test` = symptom, `y_pred` = medikament -> Vorhersage ist falsch.

18. Accuracy

Code: `accuracy_score(y_test, y_pred)`

Erklärung: Fragt: Wie viele Testfälle waren insgesamt richtig?

Beispiel: 90 von 100 richtig -> Accuracy = 90 %.

Hinweis: Bei stark unausgeglichene Klassen kann Accuracy allein irreführend sein.

19. classification_report

Code: `classification_report(y_test, y_pred)`

Erklärung: Berechnet für jede Klasse Precision, Recall, F1 und Support.

Nutzen: Deutlich aussagekräftiger als nur Accuracy.

20. Confusion Matrix

Code: `confusion_matrix(y_test, y_pred)`

Erklärung: Zeigt, welche tatsächlichen Klassen mit welchen vorhergesagten Klassen verwechselt wurden.

Nutzen: Wichtig für Error Analysis, weil du konkrete Fehlermuster erkennst.

21. Neue Texte vorhersagen

Code: vorhersagen = best_model.predict(texte)

Input: Völlig neue Texte.

Intern: Text -> TF-IDF -> bestes Modell.

Output: Eine vorhergesagte Klasse pro Text.

22. Warum prüfen wir predict_proba()?

Erklärung: LogisticRegression kann Wahrscheinlichkeiten ausgeben. LinearSVC standardmäßig nicht.

Beispiel: symptom=0.10, medikament=0.82, negation=0.08.

Folge: Wenn Logistic Regression gewinnt, kannst du eine Confidence anzeigen.

23. argmax()

Code: index = probs.argmax()

Erklärung: argmax() gibt die Position des größten Wertes zurück.

Beispiel: [0.10, 0.82, 0.08] -> größter Wert steht an Position 1.

Danach: Mit best_model.classes_[index] holst du den passenden Klassennamen.

24. Confidence Threshold

Code: confidence_threshold = 0.5

Erklärung: Nur wenn die höchste Wahrscheinlichkeit mindestens 50 % beträgt, wird die Klasse direkt ausgegeben. Sonst wird 'unsicher' ausgegeben.

Beispiel 1: 0.82 >= 0.50 -> Klasse ausgeben.

Beispiel 2: 0.41 < 0.50 -> unsicher.

Wichtig: In deiner App ist das ein Confidence-Threshold auf die höchste Klasse, kein binärer Threshold auf eine bestimmte Positivklasse.

Die komplette App in einem Satz

Zusammenfassung: Daten einlesen -> X und y trennen -> Train/Test bilden -> verschiedene Modelle und Einstellungen ausprobieren -> per Cross-Validation vergleichen -> bestes Setup auswählen -> auf unbekannten Testdaten prüfen -> Metriken berechnen -> neue Texte klassifizieren -> optional Confidence anzeigen.

Praxisbezug: Diese Reihenfolge ist für ein klassisches ML-Projekt sehr typisch.